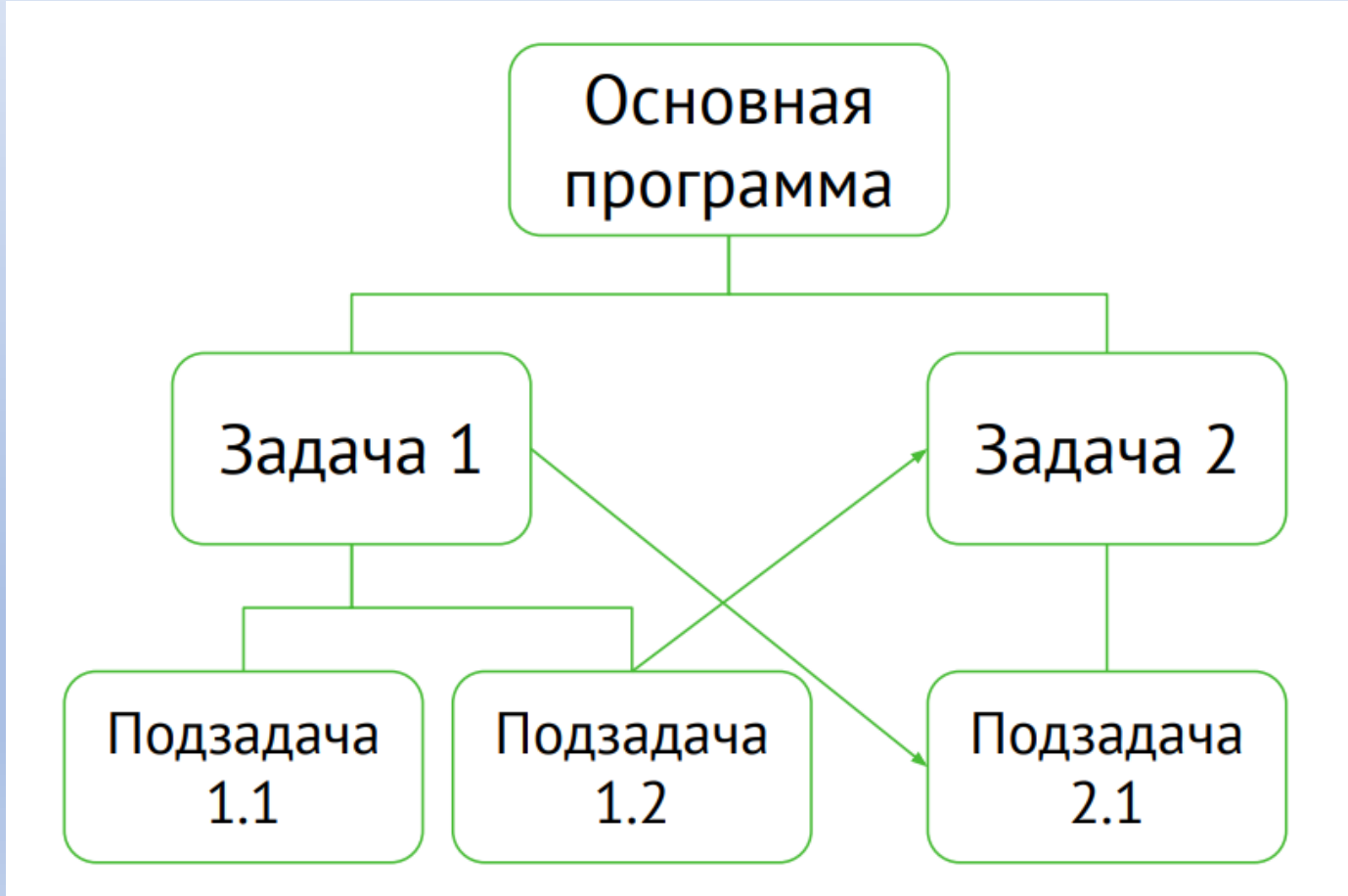
Лекция № 11
ООП. Введение в классы.

Два подхода к программированию:

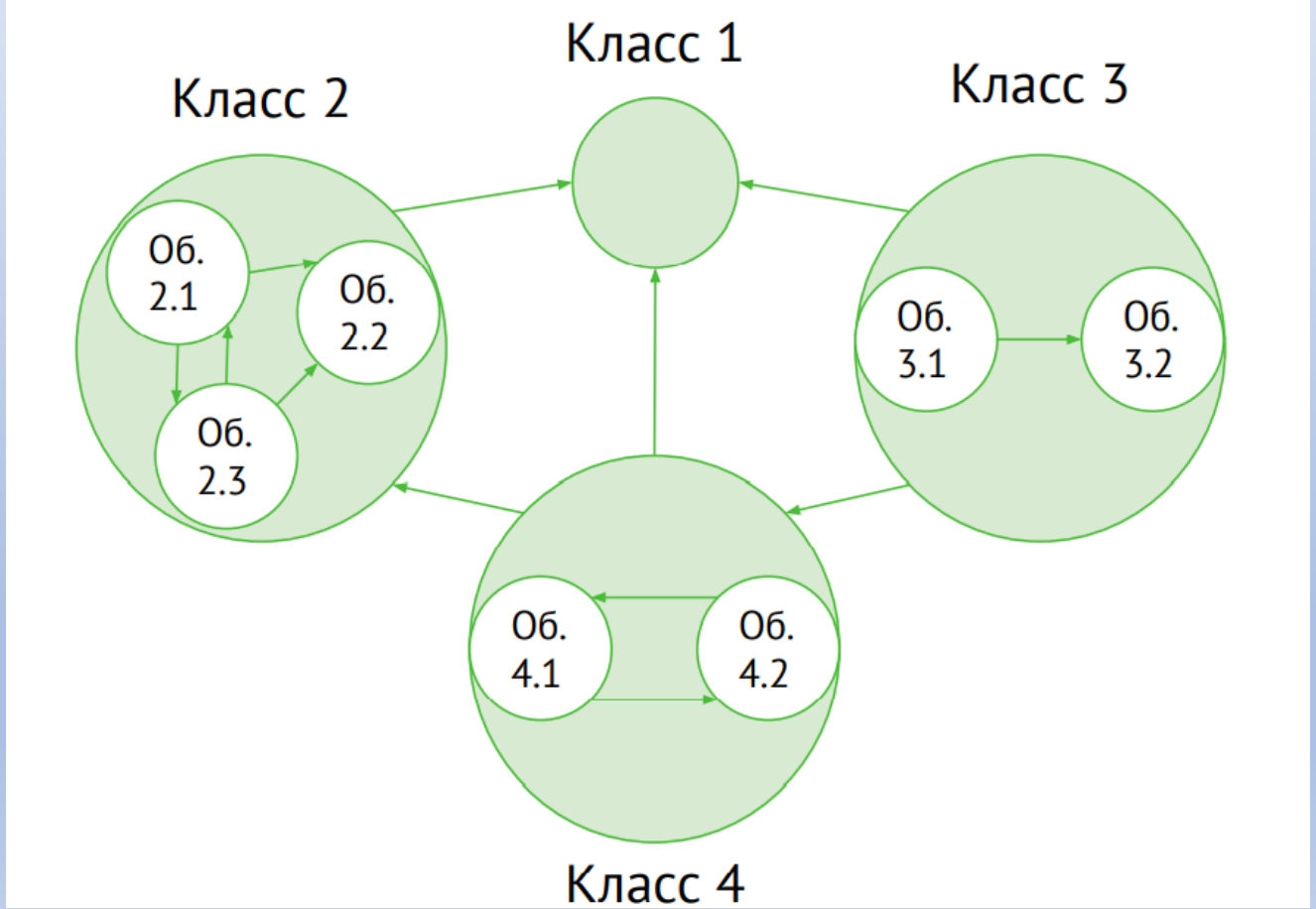
- Процедурное программирование;
- Объектно-ориентированное программирование.

- P.S: В Python реализованы обе парадигмы.

Процедурное программирование - это парадигма программирования, в которой используется линейный или нисходящий подход. Он полагается на процедуры или подпрограммы для выполнения вычислений. Программа разбивается на блоки (функции) которые вызываются в нужном порядке. Задача разбивается на несколько подзадач.

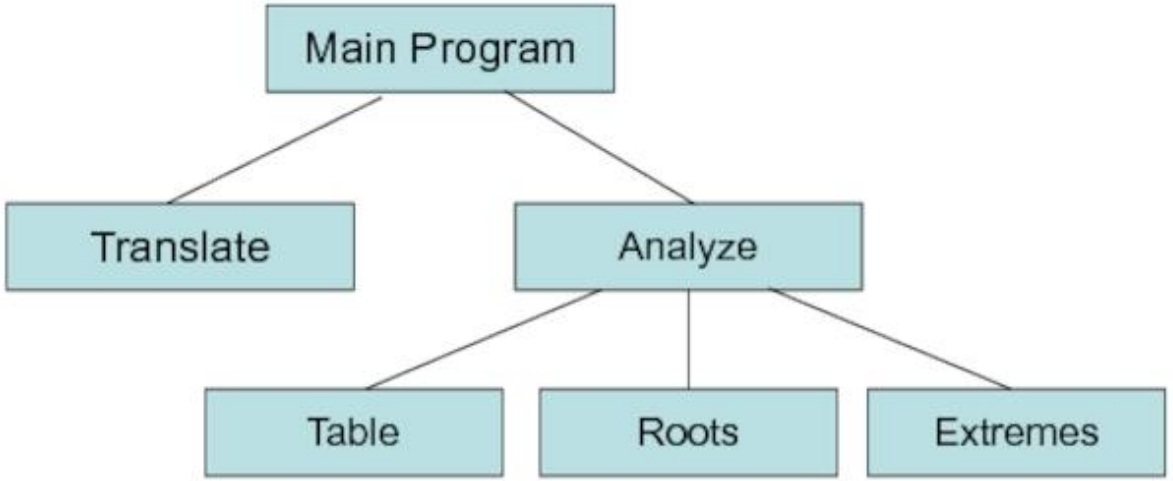


ООП - это парадигма программирования, построенная вокруг объектов. Она разделяет данные на объекты (а не на действия) и описывает содержимое и поведение объекта посредством объявления классов. Все логика разбивается не на действия, а на сущности, у которых есть общие и уникальные свойства и они могут взаимодействовать друг с другом.



- **Декомпозиция — разделение целого на части.** Также декомпозиция — это научный метод, использующий структуру задачи и позволяющий заменить решение одной большой задачи решением серии меньших задач, пусть и взаимосвязанных, но более простых.

Схема структурная программы

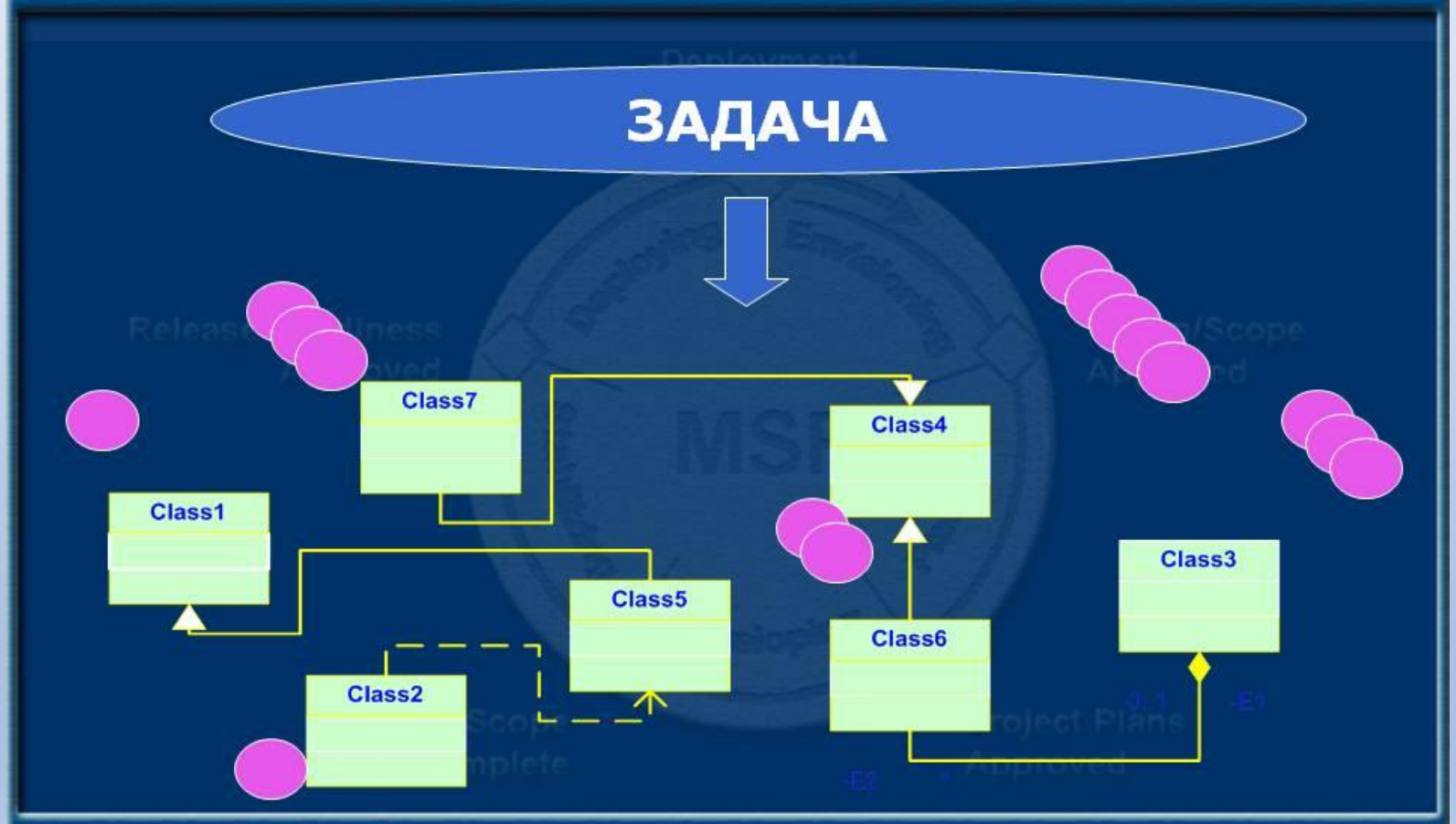


Процедурная декомпозиция – процесс разбиения программы на подпрограммы.

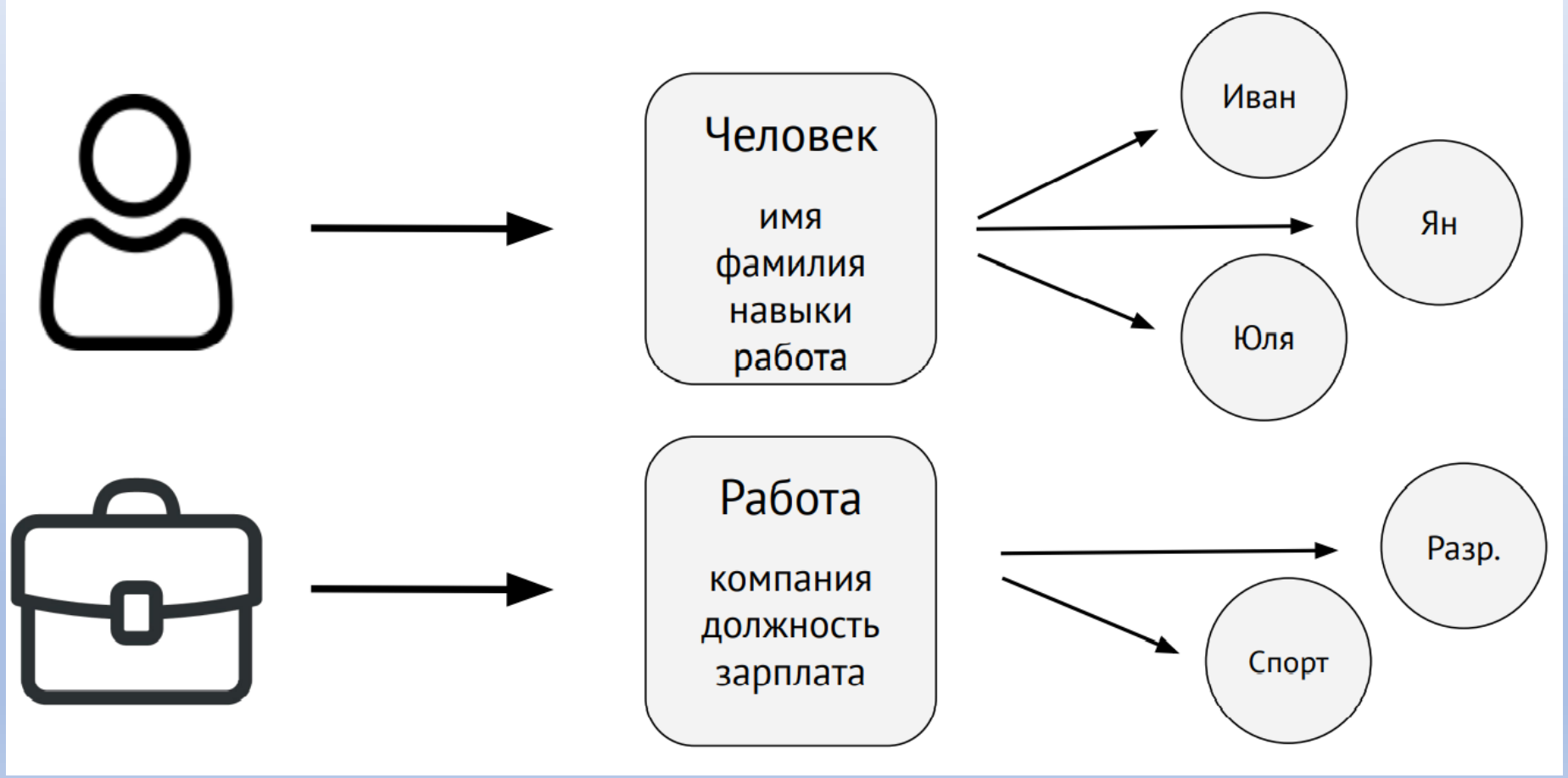
Структурной называют декомпозицию, если:

- каждая подпрограмма имеет один вход и один выход;
- подпрограммы нижних уровней не вызывают подпрограмм верхних уровней;
- размер подпрограммы не превышает 40-50 операторов.

Объектная декомпозиция

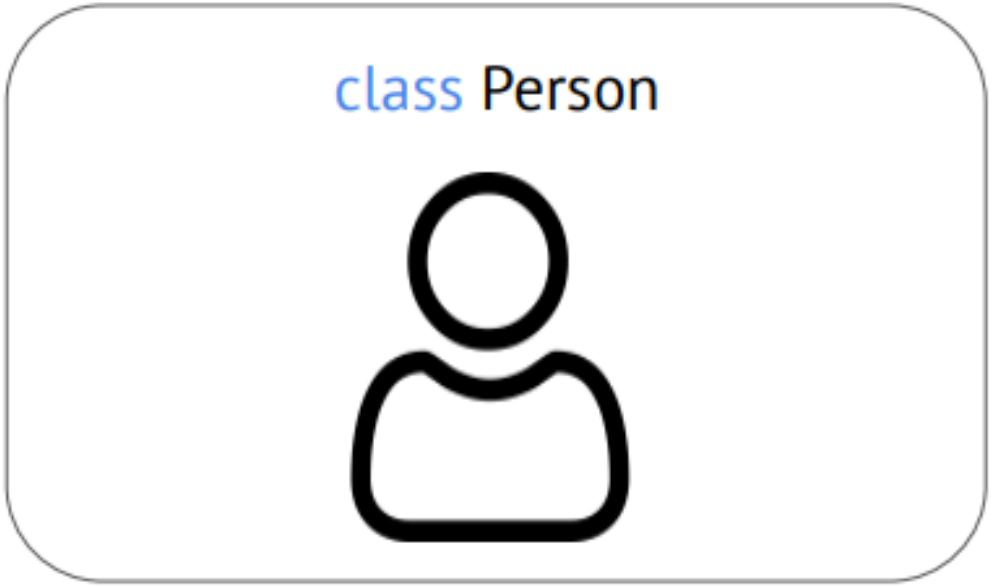


Абстракция — выделение существенных характеристик объекта, отличающих его от других объектов.



Объекты и Классы

- Класс — способ описания сущности, определяющий состояние и поведение, зависящее от этого состояния, а также правила для взаимодействия с данной сущностью.




```
class Person:
    name = …
    last_name = …

    def run(self, time):
        …
```

Объект — отдельный представитель класса, имеющий конкретное состояние и поведение, полностью определяемое классом.

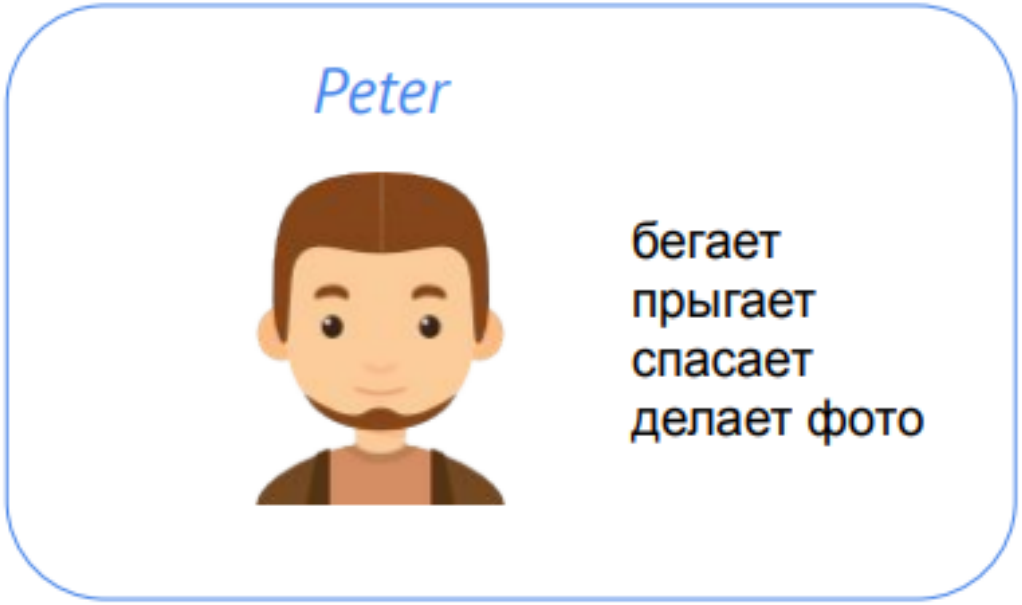
```
peter = Person()
```

Атрибуты и методы классов

- Атрибут — переменная, которая хранит некоторые данные.

```
peter = Person()
peter.name = 'Peter'
peter.last_name = 'Parker'
peter.age = 26
peter.job = 'photographer'
```

Метод — функция, с помощью которой объект может совершать действия.




```
peter = Person()
peter.run(distance=100)
peter.jump(height=10)
peter.rescue('Mary Jane')
peter.photo('Spider Man')
```

Создание своего класса

```
1  # создание класса
2  class MyClass:    # ключевое слово 'class', имя класса в CamelCase
3      x = 10        # атрибут класса
4      def func():   # атрибут класса (метод)
5          print('Hello')
6
7
8  print(MyClass.x)  # вызов атрибута 'x' класса 'MyClass' # 10
9  MyClass.func()    # вызов метода 'func' класса 'MyClass'# Hello
10 print(type(MyClass)) # <class 'type'> type - это тип всех типов,
11                      # для которых не указан явно иной метакласс
12
```

Создание экземпляра класса

```
15 exemplar = MyClass() # вызывая класс как функцию, запускается конструктор
16 print(exemplar.x)    # 10  # у экземляра класса тоже есть атрибуты
17 print(type(exemplar))  # <class '__main__.MyClass'>
18
```

```
>>> %Run lessons.py

  10
  <class '__main__.MyClass'>
```

Свойства классов:

- У класса всегда можно вызвать конструктор;
- Всегда можно обратиться к атрибутам класса.

Свойства экземпляров класса:

- Всегда можно обратиться к атрибутам класса.

Конструктор

```
24 class Register:
25     def __init__(self):  # ф-я __init__ принимает один обязательный аргумент
26                          # - сам экземпляр класса и ничего не возвращает
27         self.regist = 0
28
29
30 n = Register()
31 print(n.regist)
32 n.regist += 10
33 print(n.regist)
```

```
>>> %Run lessons.py

  0
  10
```

Методы класса

```
50 class Register:
51     def __init__(self):  # ф-я __init__ принимает один обязательный аргумент - сам экземпляр
52         self.regist = 0  # атрибут экземпляра и его значение
53     def increment(self): # объявление метода incremen
54         self.regist += 1
55     def reboot(self):    # self - наш экземпляр, обязательный параметр
56         self.regist = 0
57
58 n = Register()
59 print(n.regist)
60 n.increment()
61 print(n.regist)
62 n.reboot()
63 print(n.regist)
```

```
>>> %Run lessons.py
  0
  1
  0
```

Атрибуты класса и переменные экземпляра класса

- Переменные экземпляра класса предназначены для данных, уникальных для каждого экземпляра класса.
- Переменные класса (атрибуты данных класса) - для атрибутов и методов, общих для всех экземпляров класса.

Наследование

- Наследование используется, когда нужно, чтобы новый объект вел себя также как объект другого типа, за некоторым исключением.

```
112 class NewClass(BaseClass1, BaseClass2, BaseClass3):
113     # тело класса
114
```

Итоги:

- Мы узнали три новых типа объектов:

1. Объекты – классы (описывают сами классы)
2. Объекты – экземпляры (объекты тип которых класс)
3. Объекты – связанные методы(ссылаются на объект и на ф-ю внутри класса)

Задание на самостоятельную подготовку:

- Пространство имён (namespace)
- Область видимости (scope)
- Множественное наследование